

In mathematical notation:

$$r(h(x)) = x + 2 \neq x$$

Moving on.

Now that we have a hashing and reduction functions, what we do is create a chain of hashes and plaintexts. We pick a plaintext to start with, hash it, then use the reduction function to get another plaintext from the hash, this plaintext is again hashed, and then reduced, and then hashed, and so on for a specified number of steps.

What we have here is a chain of hashes and plaintexts that can all be derived from the starting plaintext. Such a chain could go on for thousands of iterations.

Now instead of storing each and every plaintext-hash pair along the chain, we only store the first plaintext and final hash, since all values in the middle can be computed by repeatedly using the hash and reduction function on the original plaintext.

Such a chain of say 1000 is storing information about 1000 hashes while using up only the amount of memory required for storing one plaintext-hash combination! We can now create thousands or millions of such chains and be able to cover nearly all plaintexts, while using up only a thousandth of the space.

It will still not be feasible of course to have plaintext lengths of 10-20 characters, however this method is good up to 8-9 characters – maybe even more, depending on the algorithm, the length of the chains, compression and other factors –, which depending on the hashing function, chain lengths, character set etc. can leave you with a file of a few GBs.

Let us create a chain of length 5 with a hashing function of MD5 and a reduction function which simply takes the first 6 characters of the hash as the plaintext. We will start with a randomly generated 6-digit alphanumeric plaintext:

Plaintext	:	a6tyv2
Hash	:	cd5b5e698eafeda22ab0370a88f79410
Reduced result	:	cd5b5e
Plaintext	:	cd5b5e
Hash	:	f945501a94a6836b5be04ba8fba908b0
Reduced result	:	f94550
Plaintext	:	f94550
Hash	:	e4ff786adec3a903dc3073720230117d
Reduced result	:	e4ff78